Linear time functional dynamic programming for un-regularized isotonic regression

Toby Hocking

August 18, 2026

1 Introduction

Isotonic regression is a statistical model which fits a non-decreasing function to a sequence of n data. Previous algorithms for computing this model include the Pool Adjacent Violators Algorithm (PAVA), which can be implemented in linear time, $O(n)$ [Busing, 2022]. We propose a new dynamic programming algorithm, and demonstrate the similarities and differences with PAVA.

2 Problem

2.1 Data

We assume there are n data, $y_1, \ldots, y_n \in \mathbb{R}$. In some contexts, there are also positive weights $w_1, \ldots, w_n \in \mathbb{R}_+$. When no weights are given, we take $w_i = 1$ for all i.

2.2 Isotonic regression problem

The weighted isotonic regression problem is

$$\hat{C}_n = \min_{x_1,\ldots,x_n} \sum_{i=1}^{n} w_i(x_i - y_i)^2 \text{ subject to } \forall i \in \{2,\ldots,n\},\ x_{i-1} \leq x_i. \tag{1}$$

The $\hat{C}_n \in R$ is the optimal cost, including all n data points.

3 Algorithms

3.1 Proposed dynamic programming algorithm

We propose a new algorithm based on previous work from the literature on optimal change-point detection [Hocking et al., 2020]. The idea is to use dynamic programming to recursively

compute cost functions,

$$
\begin{aligned}
C_n(x_n) &= \min_{x_1,\ldots,x_{n-1}} \sum_{i=1}^{n} w_i(x_i - y_i)^2 \text{ subject to } \forall i \in \{2,\ldots,n\},\ x_{i-1} \leq x_i. && (2)\\
&= w_n(x_n - y_n)^2 + \min_{x_{n-1}} C_{n-1}(x_{n-1}) \text{ subject to } x_{n-1} \leq x_n. && (3)\\
&= w_n(x_n - y_n)^2 + C^{\leq}_{n-1}(x_n), && (4)
\end{aligned}
$$

where $C^{\leq}_{n-1} : \mathbb{R} \to \mathbb{R}$ is a function that we call the min-less operator of the optimal cost function up to $n-1$. Dynamic programming means that we have a simple update rule for computing the optimal cost up to n data points, using the optimal cost up to $n-1$ data points. Functional representation means that the equations above use optimal cost functions rather than values.

3.2 Functional representation

Whereas previous functional dynamic programming algorithms have used either a linked list or binary tree data structure, the proposed algorithm uses arrays, which is a much simpler design that is only possible because of the problem structure. $C_i(x)$ is a convex function, defined in terms of pieces, each piece corresponding to a cluster of data with the same mean, and the min is always on the last piece (TODO examples and proof).

For each iteration of dynamic programming $i \in \{1,\ldots,n-1\}$, we therefore represent C_{i+1} as a function with $K_i + 1 \in \{1,\ldots,i+1\}$ pieces. Let $M_{i,0} = -\infty < M_{i,1} < \cdots < M_{i,K_i} < M_{i,K_i+1} = \infty$ be bounds on the function pieces, which will be computed during the dynamic programming algorithm. In the simple case with no weights, $w_i = 1$, we have

$$
\begin{aligned}
C_{i+1}(x) &= (x - y_{i+1})^2 + C^{\leq}_i(x) && (5)\\
&= \begin{cases} f_{i,1}(x) & \text{if } x \in (-\infty = M_{i,0},\ M_{i,1}],\\ & \vdots \\ f_{i,K_i+1}(x) & \text{if } x \in [M_{i,K_i},\ M_{i,K_i+1} = \infty). \end{cases} && (6)
\end{aligned}
$$

For each piece $k \in \{1,\ldots,K_i+1\}$, there is a corresponding convex quadratic function

$$f_{i,k}(x) = x^2 \hat{N}_{i,k} - 2x\hat{T}_{i,k} + \hat{c}_{i,k}, \tag{7}$$

with derivative

$$f'_{i,k}(x) = 2x\hat{N}_{i,k} - 2\hat{T}_{i,k} = 0 \Rightarrow x = \hat{T}_{i,k}/\hat{N}_{i,k}, \tag{8}$$

where $\hat{N}_{i,k}, \hat{T}_{i,k}$ are coefficients that can be efficiently computed during the algorithm, if necessary.

3.3 Details of dynamic programming update rules

For each iteration $i \in \{1, \ldots, n\}$, the dynamic programming stores the current size $K_i \in \{1, \ldots, i\}$, and arrays $N_{i,k}, T_{i,k}, M_{i,k}$, for $k \in \{1, \ldots, K_i\}$.

The initialization is $K_1 = 1$, $N_{1,1} = 1$, $T_{1,1} = y_1$, $M_{1,1} = y_1$, which is a representation of $C_1^{\leq}$.

For each iteration $i \in \{1, \ldots, n-1\}$, the dynamic programming recursively computes $C_{i+1}^{\leq}$, the next min-less function, via

$$K_{i+1} = \max\{k \in \{K_i + 1, \ldots, 1\} \mid f'_{i,k}(M_{i,k-1}) < 0 \Rightarrow \hat{T}_{i,k}/\hat{N}_{i,k} > M_{i,k-1}\}. \tag{9}$$

The condition $f'_{i,k}(M_{i,k-1}) < 0$ checks to see if the minimum occurs in the interval $(M_{i,k-1}, M_{i,k})$. The max starts by checking $k = K_i + 1$, then decrements k if the condition is not satisfied.

The coefficients of $f_{i,k}$ are computed via a cumulative sum (starting at $k = K_i + 1$ and going down),

$$\hat{N}_{i,k} = \begin{cases} 1 & \text{if } k = K_i + 1. \\ \hat{N}_{i,k+1} + N_{i,k} = 1 + \sum_{j=k}^{K_i} N_{i,j} & \text{if } k \in \{K_i, \ldots, 1\}. \end{cases} \tag{10}$$

$$\hat{T}_{i,k} = \begin{cases} y_{i+1} & \text{if } k = K_i + 1. \\ \hat{T}_{i,k+1} + T_{i,k} = y_{i+1} + \sum_{j=k}^{K_i} T_{i,j} & \text{if } k \in \{K_i, \ldots, 1\}. \end{cases} \tag{11}$$

At step $i + 1$, the new array values are

$$N_{i+1,k} = \begin{cases} N_{i,k} & \text{for } k \in \{1, \ldots, K_{i+1} - 1\}, \\ \hat{N}_{i,K_{i+1}} & \text{for } k = K_{i+1}. \end{cases} \tag{12}$$

$$T_{i+1,k} = \begin{cases} T_{i,k} & \text{for } k \in \{1, \ldots, K_{i+1} - 1\}, \\ \hat{T}_{i,K_{i+1}} & \text{for } k = K_{i+1}. \end{cases} \tag{13}$$

$$M_{i+1,k} = \begin{cases} M_{i,k} & \text{for } k \in \{1, \ldots, K_{i+1} - 1\}, \\ \hat{T}_{i,K_{i+1}}/\hat{N}_{i,K_{i+1}} & \text{for } k = K_{i+1}. \end{cases} \tag{14}$$

Note that values for all k except the largest ($k = K_{i+1}$) are the same between iterations i and $i + 1$. We therefore can avoid allocating new arrays, and simply write the new values in position K_{i+1} of the existing arrays.

3.4 Pseudocode

Algorithm 1 summarizes the solution method for the simple case without weights ($w_i = 1$ for all i).

Algorithm 1 outputs the optimal means in a run-length encoding, for a number of clusters $K \in \{1, \ldots, n\}$. Algorithm 2 can be used to convert into the solution x of problem (1), the n-vector with an optimal mean value for each input data point.

Algorithm 1 Functional dynamic programming for isotonic regression

Require: data $y_1, \ldots, y_n \in \mathbb{R}$.
  Allocate cluster sizes $N \in \mathbb{Z}^n$ and totals/means $T, M \in \mathbb{R}^n$.
  Initialize $K \leftarrow 1$, $N_1 \leftarrow 1$, $T_1 \leftarrow y_1$, $M_1 = y_1$.
  for $i = 2$ to n **do**
    $\texttt{samples} \leftarrow 1$, $\texttt{total} \leftarrow y_i$.
    while $K > 0$ and $\texttt{total}/\texttt{samples} \leq M_K$ **do**
      $\texttt{samples} \mathrel{+}= N_K$, $\texttt{total} \mathrel{+}= T_K$, $K - -$.
    end while
    $K + +$, $N_K \leftarrow \texttt{samples}$, $T_K \leftarrow \texttt{total}$, $M_K \leftarrow \texttt{total}/\texttt{samples}$.
  end for
  return K, N, M

Algorithm 2 Decoding optimal isotonic regression parameters

Require: output of Algorithm 1: number of clusters $K \in \mathbb{Z}$, sizes $N \in \mathbb{Z}^n$, means $M \in \mathbb{R}^n$.
  Allocate optimal means to output, $x \in \mathbb{R}^n$.
  Initialize $i \leftarrow n$.
  for $k = K$ to 1 **do**
    for $j = 1$ to N_k **do**
      $x_i \leftarrow M_k$, $i - -$.
    end for
  end for
  return x

3.5 Comparison with PAVA

TODO

3.6 Extensions for bounds and weights

TODO

References

F. M. T. A. Busing. Monotone regression: A simple and fast o(n) pava implementation. *Journal of Statistical Software, Code Snippets*, 102(1):1–25, 2022. doi: 10.18637/jss.v102.c01. URL `https://www.jstatsoft.org/index.php/jss/article/view/v102c01`.

T. D. Hocking, G. Rigaill, P. Fearnhead, and G. Bourque. Constrained Dynamic Programming and Supervised Penalty Learning Algorithms for Peak Detection in Genomic Data. *Journal of Machine Learning Research*, 21(87):1–40, 2020. URL `http://jmlr.org/papers/v21/18-843.html`.